

សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

មហាវិទ្យាល័យ វិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា



មុខវិជ្ជា៖

Programming Methodology in C++

មេរៀនទី១៦៖

ជំពូក ៧ ការប្រើប្រាស់ Function Template និង
Class Template

ឆ្នាំទី ១ ឆមាស ទី ២

គណៈកម្មការអភិវឌ្ឍន៍កម្មវិធីសិក្សា

© 2024

មតិកា

- ❑ 1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects
- ❑ 2. ការប្រើប្រាស់ Constructor និង Destructor
- ❑ 3. ការប្រើប្រាស់ Pointer to Class
- ❑ 4. ការប្រើប្រាស់ Class Defined with Structure , Union
- ❑ 5. ការប្រើប្រាស់ Overloading the Operators, Static Member
- ❑ 6. ការប្រើប្រាស់ FriendShip និង Inheritance
- ❑ 7. ការប្រើប្រាស់ Single និង Multiple Inheritance
- ❑ 8. ការប្រើប្រាស់ Pointer to Base Class , Abstract Base Class

មតិកា

- ❑9. ការប្រើប្រាស់ Function Template និង Class Template
- ❑10. ការប្រើប្រាស់ Virtual Member , Template Specialization
- ❑11. ការប្រើប្រាស់ NameSpace និង ដោះស្រាយ Exception
- ❑ 12. ការប្រើ ការបំប្លែងប្រភេទទិន្នន័យ
- ❑១៣. ការកំណត់ Preprocessor Directives ការប្រើ Input/ output with Files.
- ❑14. ការប្រតិបត្តិរបៀប Read និង Write Files

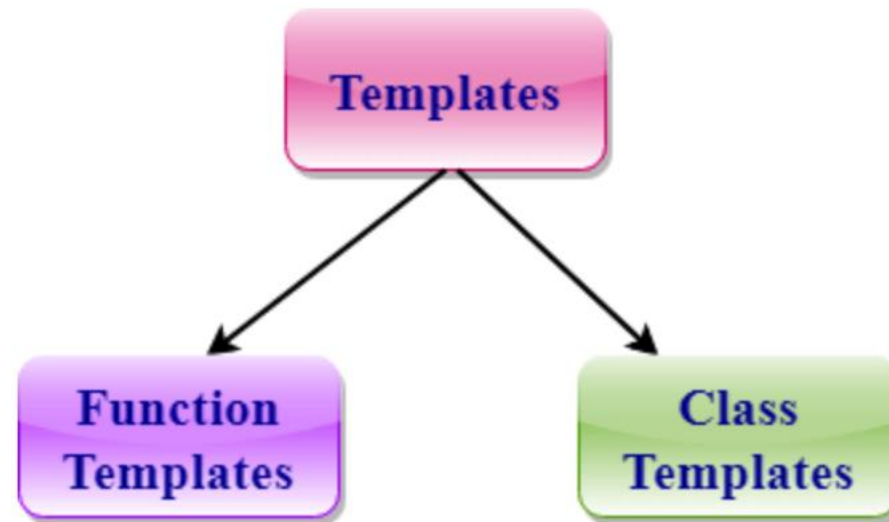
9. ការប្រើប្រាស់ Function Template និង Class Template

- 1.1. អ្វីទៅ ហៅថា Function Template ?
- 1.2. ការប្រើប្រាស់ Function Template .
- 1.3. អ្វីទៅ ហៅ ថា Class Template?
- 1.4. ការប្រើប្រាស់ Class Template.

Example : Coding .

អ្វីទៅហៅថា Template ?

Template គឺជាការសិក្សា នៃលក្ខណៈដែលបង្កើតពុម្ពគំរូ សម្រាប់ប្រើប្រាស់ដោះស្រាយបញ្ហាណាមួយដែលមានលក្ខណៈដូចគ្នា តែមានប្រភេទទិន្នន័យខុសគ្នា ។



➤ 1.១. អ្វីទៅ ហៅថា Function Template ?

- **Template Function** គឺជាប្រភេទនៃ Template ដែលគេអាចបង្កើតនូវ Function គំរូមដែលអាចប្រើប្រាស់វាបានរួម។
- **Template Function** គឺជា Function មួយដែលជាពុម្ពគំរូសម្រាប់ការងាររួមគ្នាមួយ ដែលមានលក្ខណៈដូចគ្នា តែមាន ប្រភេទទិន្នន័យខុសគ្នា ។
- **វាមានផលប្រយោជន៍ដូចជា៖** ការបង្កើត Template មួយជំនួសការសរសេរកូដដដែលៗ

1.2. ការប្រើប្រាស់ Function Template .

Syntax Template Function

```
template<class Type1, class Type2, ... >
```

```
datatypefunction_name (parameter...){ ... }
```

គោលបំណងនៃ Function Template:

- **ការប្រើប្រាស់ឡើងវិញ (Reusability):** អ្នកអាចប្រើមុខងារនេះជាមួយប្រភេទទិន្នន័យណាក៏បាន ដោយមិនចាំបាច់សរសេរកូដឡើងវិញសម្រាប់ប្រភេទនីមួយៗ។
- **ការប្រៀបធៀប (Generic Programming):** វាអាចបន្ថែមភាពសាមញ្ញ និងភាពល្អប្រើសម្រាប់ការសរសេរកូដដែលអាចដំណើរការជាមួយទិន្នន័យទាំងអស់ដោយប្រើប្រាស់ខ្លឹមសារមួយតែប៉ុណ្ណោះ។

១.3. អ្វីទៅ ហៅ ថា Class Template?

- **Class Template** គឺជា class ដែលជាពុម្ពគំរូសម្រាប់ ជំនួសឱ្យគ្រូសារ class ទាំងឡាយដែលមានលក្ខណៈដូចគ្នា ចំនួន Members ដូចគ្នា តែមាន Data Type ខុសគ្នា ។
- **Class Template** គឺជាឧបករណ៍មួយដែលអាចបង្កើត Class ដែលអាចដំណើរការជាមួយប្រភេទទិន្នន័យណាក៏បាន។ វាធ្វើឱ្យអ្នកអាចសរសេរកូដដែលអាចប្រើប្រាស់ឡើងវិញបាន ហើយបំបាត់ការសរសេរកូដដែលធ្វើសកម្មភាពដូចគ្នាដែលត្រូវតែអនុវត្តលើប្រភេទទិន្នន័យផ្សេងៗគ្នា។

➤ 1.4. ការប្រើប្រាស់ Class Template.

Syntax Template Class

template<class Type1, class Type2, ... >

```
template <class T>
class MyTemplateClass {
    // function declaration
    T member_function (T args) {
        // body
    }
    // data member
    T var_name;
};
```

គោលបំណងនៃ class Template

- ការបង្កើត Class ដែលអាចប្រើជាមួយទិន្នន័យផ្សេងៗគ្នា (Generic Class Creation)
- ការប្រើប្រាស់ឡើងវិញ (Code Reusability)
- ការសម្រួលបង្កើត Containers (Simplifying Container Creation)
- ការកាត់បន្ថយកំហុស និងការប្រើប្រាស់ត្រឹមត្រូវ (Reducing Errors and Ensuring Type Safety)
- ការបង្កើតប្រព័ន្ធទូទៅ (Generic Programming)
- បន្ថយការការងារសម្រួល (Simplifying Maintenance and Upgrades)

Coding Function Template

```

1  // Online C++ compiler to run C++ program online
2  #include <iostream>
3  using namespace std;
4
5  //Template Function
6  template<typename T>T sum(T x,T y){
7      return x+y;
8  }
9
10 int main() {
11     // Write C++ code here
12     int x1 = 12, y1=34;
13     long x2 = 123, y2=56;
14     float x3 = 58.9,y3=45.9;
15     double x4 = 789.3,y4=562.9;
16
17     //Using or calling template function sum
18     cout<<"Sum integer:" <<sum(x1,y1)<<endl;
19     cout<<"Sum long:" <<sum(x2,y2)<<endl;
20     cout<<"Sum float:" <<sum(x3,y3)<<endl;
21     cout<<"Sum Double:"<<sum(x4,y4)<<endl;
22     return 0;
23 }

```

[Success] Your code was executed successfully

Sum integer:46
Sum long:179
Sum float:104.8
Sum Double:1352.2

Custom Input

Coding Class Template

```

2  #include <iostream>
3  using namespace std;
4
5  // Step 1: Define a template
6  template <typename T>
7  class Box {
8  private:
9      T value; // The value inside the box
10 public:
11     // Constructor to initialize the box with a value
12     Box(T val) : value(val) {}
13
14     // Function to get the value from the box
15     T getValue() {
16         return value;
17     }
18
19     // Function to set a new value in the box
20     void setValue(T val) {
21         value = val;
22     }
23 };
24
25 int main() {
26     // Step 2: Create a Box for an integer
27     Box<int> intBox(123);
28     cout << "Integer Box contains: " << intBox.getValue() << endl;
29
30     // Step 3: Create a Box for a double
31     Box<double> doubleBox(45.67);
32     cout << "Double Box contains: " << doubleBox.getValue() << endl;
33
34     // Step 4: Create a Box for a string
35     Box<string> strBox("Hello, World!");
36     cout << "String Box contains: " << strBox.getValue() << endl;
37
38     // Change the value in the string box
39     strBox.setValue("New String");
40     cout << "String Box now contains: " << strBox.getValue() << endl;
41
42     return 0;
43 }

```

[Success] Your code was executed successfully

Integer Box contains: 123
 Double Box contains: 45.67
 String Box contains: Hello, World!
 String Box now contains: New String

Custom Input



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ សង្គមសេចក្តីសុខ

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>